

串口透传流程详情说明

深圳亿研电子

一、初始化流程

(1) 终端入网成功，上传自己的短地址到协调器

```
00217:          // Start sending the periodic message in a regular interval.
00218:          HalLedSet (HAL_LED_1, HAL_LED_MODE_ON);
00219:
00220:          if (SampleApp_NwkState != DEV_ZB_COORD)
00221:              SerialApp_DeviceConnect();
00222:      }
00223:      else
00224:      {

00441: void SerialApp_DeviceConnect()
00442: {
00443:     #if ZDO_COORDINATOR
00444:
00445:     #else
00446:
00447:         uint16 nwkAddr;
00448:         uint16 parentNwkAddr;
00449:         char buff[30] = {0};
00450:
00451:         HalLedBlink ( HAL_LED_2, 3, 50, (1000 / 4) );
00452:         nwkAddr = NLME_GetShortAddr();
00453:         parentNwkAddr = NLME_GetCoordShortAddr();
00454:         sprintf(buff, "parent:%d self:%d\r\n", parentNwkAddr, nwkAddr);
00455:         HalUARTWrite ( 0, (uint8*)buff, strlen(buff));
00456:
00457:         SerialApp_TxAddr.addrMode = (afAddrMode_t)Addr16Bit;
00458:         SerialApp_TxAddr.endPoint = SERIALAPP_ENDPOINT;
00459:         SerialApp_TxAddr.addr.shortAddr = parentNwkAddr;
00460:
00461:         buff[0] = HI_UINT16 ( nwkAddr );
00462:         buff[1] = LO_UINT16 ( nwkAddr );
00463:
00464:         if ( AF_DataRequest ( &SerialApp_TxAddr, &SerialApp_epDesc,
00465:                               SERIALAPP_CONNECTREQ_CLUSTER,
00466:                               2,
00467:                               (uint8*)buff,
00468:                               &SerialApp_MsgID,
00469:                               0,
00470:                               AF_DEFAULT_RADIUS ) == afStatus_SUCCESS )
00471:         {
00472:         }
00473:         else
00474:         {
00475:         }
```

终端串口输出：

parent:0,xxxx

其中 xxxx 为终端的短地址。

(2) 协调器接收终端上传的短地址

协调器收到终端的短地址后，把地址保存在 SerialApp_TxAddr 里，SerialApp_TxAddr 用于协调器往终端点播数据。同时把自己的短地址(协调器为 0)发给终端。

```
00329:
00330:     case SERIALAPP_CONNECTREQ_CLUSTER:
00331:         SerialApp_ConnectReqProcess ((uint8*)pkt->cmd.Data);
00332:
```

```

00496: void SerialApp_ConnectReqProcess(uint8 *buf)
00497: {
00498:     uint16 nwkAddr;
00499:     char buff[30] = {0};
00500:
00501:     SerialApp_TxAddr.addrMode = (afAddrMode_t)Addr16Bit;
00502:     SerialApp_TxAddr.endPoint = SERIALAPP_ENDPOINT;
00503:     SerialApp_TxAddr.addr.shortAddr = BUILD_UINT16(buf[1], buf[0]);
00504:     nwkAddr = NLME_GetShortAddr();
00505:
00506:     sprintf(buff, "self:%d  child:%d\r\n", nwkAddr, SerialApp_TxAddr.addr.shortAddr);
00507:     HalUARTWrite ( 0, (uint8*)buff, strlen(buff));
00508:
00509:     buff[0] = HI_UINT16( nwkAddr );
00510:     buff[1] = LO_UINT16( nwkAddr );
00511:
00512:     if ( AF_DataRequest( &SerialApp_TxAddr, &SerialApp_epDesc,
00513:                         SERIALAPP_CONNECTRSP_CLUSTER,
00514:                         2,
00515:                         (uint8*)buff,
00516:                         &SerialApp_MsgID,
00517:                         0,
00518:                         AF_DEFAULT_RADIUS ) == afStatus_SUCCESS )
00519:     {
00520:     }
00521:     else
00522:     {
00523:         // Error occurred in request to send.
00524:     }
00525:
00526:     HalLedSet(HAL_LED_2, HAL_LED_MODE_ON);
00527:     HalUARTWrite ( 0, "< connect success>\n", 23);
00528: } ? end SerialApp_ConnectReqProcess ?

```

协调器串口输出:

self:0,xxxx

< connect success >

其中 xxxx 为终端的短地址。

(3) 终端接收协调器的地址

终端收到协调器地址后，保存在结构 SerialApp_TxAddr 中，结构 SerialApp_TxAddr 用于终端往协调器发点播数据。

```

00482: void SerialApp_DeviceConnectRsp(uint8 *buf)
00483: {
00484:     #if ZDO_COORDINATOR
00485:
00486:     #else
00487:         SerialApp_TxAddr.addrMode = (afAddrMode_t)Addr16Bit;
00488:         SerialApp_TxAddr.endPoint = SERIALAPP_ENDPOINT;
00489:         SerialApp_TxAddr.addr.shortAddr = BUILD_UINT16(buf[1], buf[0]);
00490:
00491:         HalLedSet(HAL_LED_2, HAL_LED_MODE_ON);
00492:         HalUARTWrite ( 0, "< connect success>\n", 23);
00493:     #endif
00494: }

```

终端串口输出:

< connect success >

(4) 到这里初始化就完成了，不管协调器，还是终端的串口接收到数据，都会发给对方了。

二、数据发送接收流程

(1) 触发数据的第一步，不管是终端还是协调器，就是串口收到数据。串口回调函数 SerialApp_CallBack 调用的是 SerialApp_Send 去收数据。

```

00424: static void SerialApp_Callback(uint8 port, uint8 event)
00425: {
00426:     (void)port;
00427:
00428:     if ((event & (HAL_UART_RX_FULL | HAL_UART_RX_ABOUT_FULL | HAL_UART_RX_TIMEOUT)) &&
00429:         #if SERIAL_APP_LOOPBACK
00430:         (SerialApp_TxLen < SERIAL_APP_TX_MAX) )
00431:     #else
00432:         !SerialApp_TxLen)
00433:     #endif
00434:     {
00435:         SerialApp_Send();
00436:     }
00437: }
00438:

```

```

00350: static void SerialApp_Send(void)
00351: {
00352:     #if SERIAL_APP_LOOPBACK
00353:         if (SerialApp_TxLen < SERIAL_APP_TX_MAX)
00354:         {
00355:             SerialApp_TxLen += HalUARTRead(SERIAL_APP_PORT, SerialApp_TxBuf+SerialApp_TxLen+1,
00356:                                             SERIAL_APP_TX_MAX-SerialApp_TxLen);
00357:         }
00358:
00359:         if (SerialApp_TxLen)
00360:         {
00361:             (void)SerialApp_TxAddr;
00362:             if (HalUARTWrite(SERIAL_APP_PORT, SerialApp_TxBuf+1, SerialApp_TxLen))
00363:             {
00364:                 SerialApp_TxLen = 0;
00365:             }
00366:             else
00367:             {
00368:                 osal_set_event(SerialApp_TaskID, SERIALAPP_SEND_EVT);
00369:             }
00370:         }
00371:     #else
00372:         if (!SerialApp_TxLen &&
00373:             (SerialApp_TxLen = HalUARTRead(SERIAL_APP_PORT, SerialApp_TxBuf+1, SERIAL_APP_TX_MAX)))
00374:         {
00375:             // Pre-pend sequence number to the Tx message.
00376:             SerialApp_TxBuf[0] = ++SerialApp_TxSeq;
00377:         }
00378:
00379:         if (SerialApp_TxLen)
00380:         {
00381:             if (afStatus_SUCCESS != AF_DataRequest(&SerialApp_TxAddr,
00382:                                                  (EndPointDesc_t *)&SerialApp_epDesc,
00383:                                                  SERIALAPP_CLUSTERID1,
00384:                                                  SerialApp_TxLen+1, SerialApp_TxBuf,
00385:                                                  &SerialApp_MsgID, 0, AF_DEFAULT_RADIUS))
00386:             {
00387:                 osal_set_event(SerialApp_TaskID, SERIALAPP_SEND_EVT);
00388:             }
00389:         }
00390:     #endif
00391: } // end SerialApp_Send

```

第 372 行: SerialApp_TxLen 默认为 0, 表示可以调 HalUARTRead() 去读串口数据。

第 373 行: 调函数 HalUARTRead 去读串口数据。

第 376 行: 如果从串口读到数据, 在第一个字节增加序列号 ++SerialApp_TxSeq, SerialApp_TxSeq 默认为 0, 所以这里第一次发送的序列号为 1。

第 381 行: 调函数 AF_DataRequest 去无线发送到协调器(或者终端或者路由)。

第 387 行: 如果调函数 AF_DataRequest 返回失败, 调用事件 SERIALAPP_SEND_EVT 去触发函数 SerialApp_Send()。

(2) 对方接收数据(协调器或者终端)

```

00271:
00272:     switch ( pkt->clusterId )
00273:     {
00274:         // A message with a serial data block to be transmitted on the serial port.
00275:         case SERIALAPP_CLUSTERID1: //收到发送过来的数据通过串口输出到电脑显示
00276:             // Store the address for sending and retrying.
00277:             osal_memcpy(&SerialApp_RxAddr, &(pkt->srcAddr), sizeof( afAddrType_t ));
00278:
00279:             seqnb = pkt->cmd.Data[0];
00280:
00281:             // Keep message if not a repeat packet
00282:             if ( (seqnb > SerialApp_RxSeq) || // Normal
00283:                 ((seqnb < 0x80) && ( SerialApp_RxSeq > 0x80)) ) // Wrap-around
00284:             {
00285:                 // Transmit the data on the serial port. // 通过串口发送数据到PC机
00286:                 if ( HalUARTWrite( SERIAL_APP_PORT, pkt->cmd.Data+1, (pkt->cmd.DataLength-1) ) )
00287:                 {
00288:                     // Save for next incoming message
00289:                     SerialApp_RxSeq = seqnb;
00290:                     stat = OTA_SUCCESS;
00291:                 }
00292:                 else
00293:                 {
00294:                     stat = OTA_SER_BUSY;
00295:                 }
00296:             }
00297:             else
00298:             {
00299:                 stat = OTA_DUP_MSG;
00300:             }
00301:
00302:             // Select appropriate OTA flow-control delay.
00303:             delay = (stat == OTA_SER_BUSY) ? SERIALAPP_NAK_DELAY : SERIALAPP_ACK_DELAY;
00304:
00305:             // Build & send OTA response message.
00306:             SerialApp_RspBuf[0] = stat;
00307:             SerialApp_RspBuf[1] = seqnb;
00308:             SerialApp_RspBuf[2] = LO_UINT16( delay );
00309:             SerialApp_RspBuf[3] = HI_UINT16( delay );
00310:             osal_set_event( SerialApp_TaskID, SERIALAPP_RESP_EVT ); //收到数据后，发送一个响应事件
00311:             osal_stop_timerEx(SerialApp_TaskID, SERIALAPP_RESP_EVT);
00312:             break;

```

第 279 行：取出收到的序列号 seqnb，根据第 1 步的第 376 行，知道 seqnb 为 1。

第 282 行：SerialApp_RxSeq 默认为 0xC3，即这行不成立。

第 283 行：SerialApp_RxSeq 默认为 0xC3，成立。

第 286 行：串口输出收到的数据。

第 289 行：SerialApp_RxSeq 保留发送方的序列号，即 SerialApp_RxSeq=1。

第 310 行：触发响应的事件 SERIALAPP_RESP_EVT，其中响应的内容里 SerialApp_RspBuf[1] 带有接收到的数据包序列号 1。

(3) 上面的 310 行的事件处理如下。

```

00245:     if ( events & SERIALAPP_RESP_EVT )
00246:     {
00247:         SerialApp_Resp();
00248:         return ( events ^ SERIALAPP_RESP_EVT );
00249:     }
00250:

```

```

00402: static void SerialApp_Resp(void)
00403: {
00404:     if (afStatus_SUCCESS != AF_DataRequest(&SerialApp_RxAddr,
00405:                                             (EndPointDesc_t *)&SerialApp_epDesc,
00406:                                             SERIALAPP_CLUSTERID2,
00407:                                             SERIAL_APP_RSP_CNT, SerialApp_RspBuf,
00408:                                             &SerialApp_MsgID, 0, AF_DEFAULT_RADIUS))
00409:     {
00410:         osal_set_event(SerialApp_TaskID, SERIALAPP_RESP_EVT);
00411:     }
00412: }

```

第 404 行，调用 AF_DataRequest 返回收到的序列号到发送方。

(4) 发送方处理响应

```

00313:
00314: // A response to a received serial data block. // 接到响应消息
00315: case SERIALAPP_CLUSTERID2:
00316:     if ((pkt->cmd.Data[1] == SerialApp_TxSeq) &&
00317:         ((pkt->cmd.Data[0] == OTA_SUCCESS) || (pkt->cmd.Data[0] == OTA_DUP_MSG)))
00318:     {
00319:         SerialApp_TxLen = 0;
00320:         osal_stop_timerEx(SerialApp_TaskID, SERIALAPP_SEND_EVT);
00321:     }
00322:     else
00323:     {
00324:         // Re-start timeout according to delay sent from other device.
00325:         delay = BUILD_UINT16( pkt->cmd.Data[2], pkt->cmd.Data[3] );
00326:         osal_start_timerEx( SerialApp_TaskID, SERIALAPP_SEND_EVT, delay );
00327:     }
00328:     break;
00329:

```

第 316 行: SerialApp_TxSeq 现在还是 1, 而响应的数据 pkt->cmd.Data[1]也是 1。

第 319 行: SerialApp_TxLen 置 0。

(5) 如果串口还有数据, 重新从第 1 步开始, 因为上面的第(4)步的第 319 行, 已经把 SerialApp_TxLen 置 0。

(6) 第二次串口接收数据的时候, 发送的序列为 SerialApp_TxSeq 为 2。上面第(2)步的 282 行成立(即 if(2>1))

(7) 依次类推。。。